Código:FT-DG-002
Versión:001
Fecha:08/06/2021

FORMATO DE DESARROLLO E INTEGRACIONES

Originacion Aplicación Admin

STF GROUP S.A.
INFORMATICA – Py. CREDI10 SOFTWARE PROPIO

Cali, Viernes, 17 de noviembre de 2023

TABLA DE CONTENIDO

INTRODUCCION 3
1. DETALLE TECNICO 4
1.1 Detallar componentes existentes..... 4
1.2 Detallar funcionalidades 5
1.3 Observaciones y/o Comentarios.....**¡Error! Marcador no definido.**
2. CASOS DE PRUEBAS UNITARIAS
¡Error! Marcador no definido.

INTRODUCCION

Este documento muestra en detalle todas las actividades de la solución a desarrollar con sus dependencias, entradas y salidas. Así como los casos de pruebas unitarias realizadas a la solución.

NOTA: El desarrollo de cualquier solución de requerimiento debe seguir los estándares de desarrollo que define Arquitectura.

1. DETALLE TECNICO

1.1 Descripción del Requerimiento:

Realizar la primera parte del proceso de inscripción del cliente y se realiza una serie de validaciones de seguridad para asignar un cupo de crédito por un valor en pesos colombianos.

1. Capturar los datos básicos del cliente para la validación del ejercicio
2. Realizar validaciones de Inspektor, Datacredito
3. Asignar un cupo de crédito de acuerdo a las políticas

Ver el link: RF-ADM- 001 - FT-SL-001-Módulo_Originación.docx

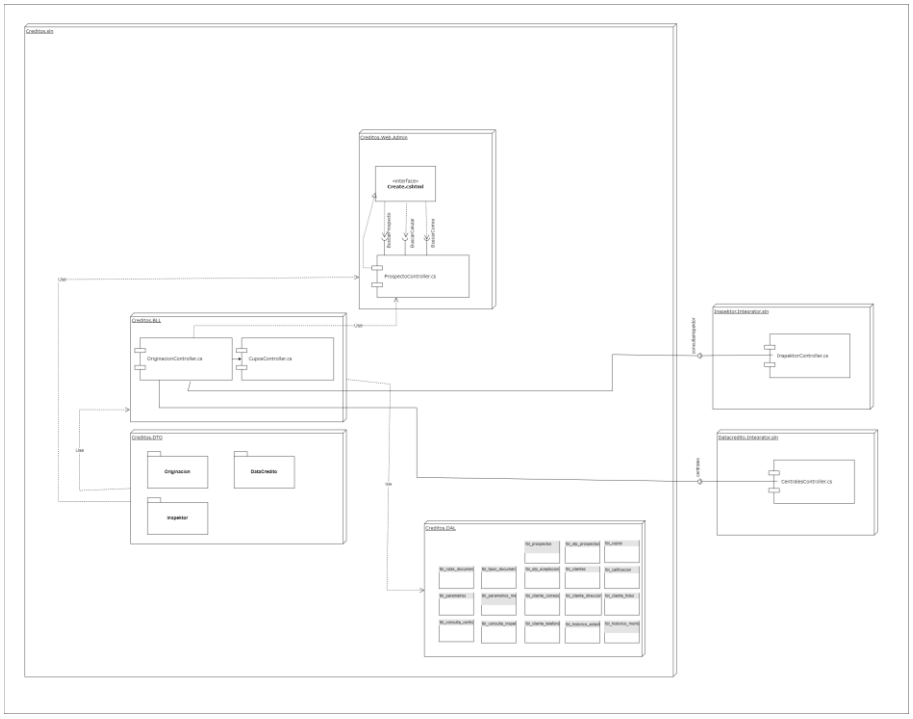
1.2 Detallar componentes existentes

Creditos.Web.Admin • Controladores ○ ProspectoController.cs ○ CupoController.cs • Vistas ○ Create.cshtml ○ Header.cshtml ○ Footer.cshtml	Clases y vistas implementadas para cumplir con el propósito de registrar un prospecto y crear el cupo.
Creditos.BLL • Prospectos.cs • CuposController.cs • OriginacionController.cs • ServiciosController.cs • InspektorServices.cs • CentralesServices.cs • NotificationController.cs • SecurityUtil.cs • RestUtil.cs	Clases implementadas para realizar las consultas a DataCredito, Inspecktor, validaciones de identidad como: celular, correo, fecha expedición, numero de celular, correo.
Creditos.DAL • ModelC10 ○ C10Model.edmx	Modelo de datos de entidad.
Creditos.DTO • CentralesRequest • CentralesLoginRequest • InspektorLoginRequest • InspektorRequest • SolicitudFormularioDTO • AceptacionReferenciasDTO • HabeasDataDTO • PagareDTO	Clases de transferencia de datos
StyleLoader.css ,	Estilos pantalla carga

StylesRegistro.css ,	Estilos vista registro
AutoNumeric	Librería Formato de números
Notification.js	Contiene los métodos que se utilizan para mostrar las alertas a los usuarios.
BASE DE DATOS	
C10_Credi10NET.dbo.tbl_calificacion	Tablas
C10_Credi10NET.dbo.tbl_cliente_correos	Tablas
C10_Credi10NET.dbo.tbl_cliente_direcciones	Tablas
C10_Credi10NET.dbo.tbl_cliente_fotos	Tablas
C10_Credi10NET.dbo.tbl_cliente_telefonos	Tablas
C10_Credi10NET.dbo.tbl_clientes	Tablas
C10_Credi10NET.dbo.tbl_cupos	Tablas
C10_Credi10NET.dbo.tbl_historico_estados_cupos	Tablas
C10_Credi10NET.dbo.tbl_historico_monto_cupos	Tablas
C10_Credi10NET.dbo.tbl_otp_aceptacion	Tablas
C10_Credi10NET.dbo.tbl_otp_prospectos	Tablas
C10_Credi10NET.dbo.tbl_parametros	Tablas
C10_Credi10NET.dbo.tbl_parametros_mensajes	Tablas
C10_Credi10NET.dbo.tbl_prospectos	Tablas
C10_Credi10NET.dbo.tbl_rutas_documentos	Tablas
C10_Credi10NET.dbo.tbl_tipos_documentos	Tablas
C10_Datacredito.dbo.tbl_consulta_centrales	Tablas
C10_Inspektor.dbo.tbl_consulta_inspektor	Tablas

1.3 Detallar funcionalidades

1.3.1 Diagrama de Componentes



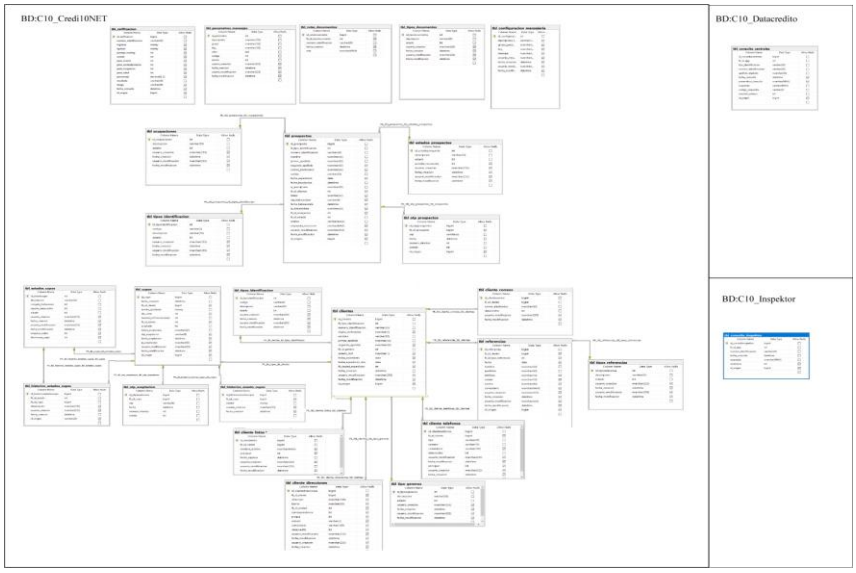
1.3.2 MER

Comentado [HA1]: Usar para estos manuales, mejor diagramas de componentes ya que por ser funcionalidades internas los diagramas de arquitectura dan una visualizacion muy general

Opcional usar: https://app.diagrams.net/

Comentado [HA2R1]: https://www.youtube.com/watch?app=desktop&v=JF7bH5Xo6ys

Comentado [HA3R1]: https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/diagrama-de-componentes/



1.3.3 Parametrizaciones

Parametrizaciones desde base de datos.

- Tbl_parametros, Campo Key, Valor EncendidoInspektor: Sirve para habilitar o deshabilitar la consulta del prospecto en inspecktor, si está en True se consulta, si está en False se omite la consulta.

- Tbl_parametros, Campo Key, Valor EncendidoSMS: Sirve para encender o apagar el envió de sms con el otp de aceptación del cupo Asi: ”Para continuar con la solicitud su otp es el siguiente {0}.” si esta en True, se notifica por sms, si esta en false no.

- Tbl_parametros, Campo Key, Valor EncendidoValidacionCorreo: Sirve para encender o apagar la validación del correo electrónico en el proceso de originacion, si está en true se valida el correo electrónico, si está en false se omite la validacion

- Los mensajes que se muestran en las vistas durante el proceso de originacion están parametrizados en la tabla *[tbl_parametros_mensajes]* y tiene configurado en la columna ‘*grupo’* la palabra ‘Originación’, los mensajes que se muestran en este proceso a continuación:

id_parametro	descripcion	grupo	key_	valor	codigo
1	Ya acepto Habeas (HabeasDataAceptado)	Originación	HabeasAceptado	Ya acepto el HabeasData, el estado del prospecto...	1001
2	No acepto Habeas data (HabeasDataAceptado)	Originación	HabeasNoAceptado	No acepto el HabeasData, el estado del prospecto...	1002
3	Otp validado con exito, respuesta al correo (Val...	Originación	RespuestaAlCorreo	La respuesta a la solicitud de cupo, fue enviada a t...	1003
4	Generando nuevo OTP (EnviarCorreoSolicitud)	Originación	NuevoOtpProspecto	Se creó un nuevo código por favor revise su corre...	1004
5	Supero limite de intentos (ValidarOtpsCreados)	Originación	LimiteMaximoIntentos	Se ha superado el límite máximo de intentos, el pro...	1005
6	Solicitud aprobada (AceptacionHabeasData)	Originación	InformaSolicitudAprobada	La respuesta a la solicitud de cupo, fue enviada a t...	1006
7	Solicitud fallo (AceptacionHabeasData)	Originación	InformaFalloSolicitud	La respuesta a la solicitud de cupo, fue enviada a t...	1007
8	Solicitud rechazada (AceptacionHabeasData)	Originación	InfomaSolicitudRechazada	La respuesta a la solicitud de cupo, fue enviada a t...	1008
9	Otp Incorrecto intentos (ValidarOtpFlujoCajera)	Originación	OtpIncorrectoIntentos	El OTP es incorrecto, tiene {0} intentos mas	1009
10	Otp Incorrecto (ValidarOtpFlujoCajera)	Originación	OtpIncorrecto	El OTP es incorrecto	1010
11	TokenNoExiste	Originación	TokenNoExiste	El token no existe por favor comunicarse con nues...	1011
12	Aviso referencias diferentes (ValidarOtpAcepta...	Originación	ReferenciasDiferentes	La referencia personal y familiar debe ser una pers...	1012
13	Aviso celular referencias diferentes (ValidarOtp...	Originación	CelularReferenciasDifere...	El celular de la referencia personal y familiar deben ...	1013
14	Otp Incorrecto aceptacion cupo (ValidarOtpAc...	Originación	OtpAceptacionIncorrecto	El otp no coincide	1014
15	Aviso ya puede comprar (OtpAceptacionCoinci...	Originación	EmpezarCompras	Ya puedes empezar a comprar con tu cupo en las t...	1015
59	Excepcion del metodo CrearCupo	Originación	ErrorCreandoCupo	Error al crear cupo	2044
61	Advertencia ya aceptó el cupo	Originación	YaAceptoElCupoEstado	Ya acepto el Cupo, el estado del cupo es:	2046
62	Advertencia NO aceptó el cupo	Originación	NoAceptoElCupoEstado	No acepto el Cupo, el estado del cupo es:	2047
112	El celular de las referencias debe ser diferente ...	Originación	CelularReferenciaDiferen...	El celular de las referencias debe ser diferente al c...	1018

1.3.4 Proceso

El presente requerimiento se desarrolla bajo la solución Creditos.sln, de esta solución los proyectos que abarcan el presente requerimiento son cuatro Creditos.Web.Originacion, Creditos.BLL, Creditos.DAL y Creditos.DTO.

Es importante mencionar que en el proceso de Originacion existen validaciones y consumos que pueden ser apagados o encendidos con el fin de omitir estos mismos, a continuación, se muestran las validaciones y consumos que están parametrizados y pueden tener un valor de “True” o “False” según la necesidad.

Tbl_parametros	
Key_	Descripción
EncendidoInspektor	Sirve para habilitar o deshabilitar la consulta del prospecto en inspecktor, si está en True se consulta, si está en False se omite la consulta.
EncendidoSMS	Sirve para encender o apagar el envió de sms con el otp de aceptación del cupo Asi: ”Para continuar con la solicitud su otp es el siguiente {0}.” si esta en True, se notifica por sms, si esta en false no.
EncendidoValidacionCorreo	Sirve para encender o apagar la validación del correo electrónico en el proceso de originacion, si está en true se valida el correo electrónico, si está en false se omite la validacion.

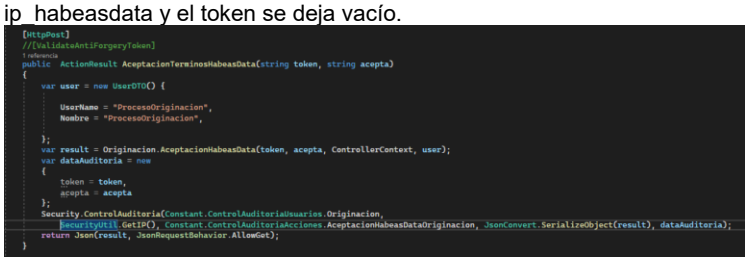
El proceso inicia con la inscripción de prospectos en Create.cshtml que contiene el formulario de registro y las funcionalidades de código JavaScript; Las funcionalidades de JS mencionadas son el consumo de endpoints con Ajax que realizan las validaciones de datos únicos del formulario para que no se repitan: cedulas, celular, correo electrónico en *[tbl_prospectos]* y *[tbl_clientes]*; los métodos que se consumen son de

ProspectoController y corresponden a *BuscarProspecto()*, *BuscarCelular()*, *BuscarCorreo()*, en caso de que los datos ingresados en el formulario ya estén registrados se muestra una alerta.

Una vez diligenciado el formulario y realizada la acción clic sobre el botón de enviar OTP, se envía la información del formulario en el objeto SolicitudFormularioDTO y se inserta en *[tbl_prospectos]* la información suministrada, se crea un token alfanumerico y se crea un OTP en la tabla *[tbl_otp_prospectos]*, el OTP generado y el token se envía por correo electrónico junto con una plantilla HTML que esta alojada en BD en la tabla parámetros bajo el key_ "*EmailMessageSolicitud*" él envió del correo se realiza haciendo uso del método *EnviarCorreoSolicitud()* del controlador *NotificationController.cs*

Al ingresar el OTP en la pantalla y se da en culminar registro se consume el endpoint *ValidarOtpProspecto()* si el OTP no coincide se muestra el mensaje correspondiente (*[tbl_parametros_mensajes]* ***key_*** -> *OtpIncorrecto*) se suma el número de intentos y en caso de que se supere el número de intentos que son tres se cambia el OTP. Si el OTP coincide, en la respuesta del Ajax que consume *ValidarOtpProspecto()* se vuelve a realizar una petición POST con Ajax al endpoint *AceptacionTerminosHabeasData (string token, string acepta),* con el token del prospecto y el parametro acepta setteado como "Si".

Al endpoint *AceptacionTerminosHabeasData()* llega "Si" entonces se actualiza el prospecto con los datos de la aceptación del tratamiento de datos: el OTP, fecha_habeasdata, ip_habeasdata y el token se deja vacío.



```
[HttpPost]
//[ValidateAntiForgeryToken]
1 referencia
public  ActionResult AceptacionTerminosHabeasData(string token, string acepta)
{
    var user = new UserDTO() {

        UserName = "ProcesoOriginacion",
        Nombre = "ProcesoOriginacion",

    };
    var result = Originacion.AceptacionHabeasData(token, acepta, ControllerContext, user);
    var dataAuditoria = new
    {
        token = token,
        acepta = acepta
    };
    Security.ControlAuditoria(Constant.ControlAuditoriaUsuarios.Originacion,
        SecurityUtil.GetIPC(), Constant.ControlAuditoriaAcciones.AceptacionHabeasDataOriginacion, JsonConvert.SerializeObject(result), dataAuditoria);
    return Json(result, JsonRequestBehavior.AllowGet);
}
```

A continuación, se crea el documento pdf llamando el método *GenerarPdfHabeasData()* perteneciente a *ReportesController* y recibe el objeto *HabeasDataDTO*. En *GenerarPdfHabeasData* se usa rotativa para generar el archivo pdf basado en la vista HabeasData.cshtml.

Cuando se ingresa el otp en la misma vista del formulario de registro para aceptar el tratamiento de datos se procede a crear el archivo pdf; para almacenar el pdf se usa una ruta que es construida a partir de lo parametrizado en la *[tbl_parametros]*, de esta tabla se toma la columna *value*, de los *key_* RutaArchivosClientes y CarpetaActualArchivosClientesSemestre a los cuales se les concatena la cedula del prospecto encriptada con sha256(), por lo que la ubicación de los archivos está dada por: RutaArchivosClientes + CarpetaActualArchivosClientesSemestre + "\\" + Sha256(cedulaProspecto) .

Una vez creado el documento del HabeasData, se procede con las consultas a Inspecktor y centrales de riesgo, para esto se hace uso del controlador ServiciosController.cs que aloja los métodos *ConsultaInspektor()* que consume el api de inspeckor y *ConsultaCentralesDeRiesgo()* que consume el api de datacredito.

ConsultaInspektorCentrales () se encarga de realizar las dos consultas para el prospecto: Inspecktor, Datacredito y adicionalmente crea el cliente y el cupo en caso de obtener una respuesta satisfactoria de ambas consultas. Tiene como finalidad ejecutar el consumo y consulta de ambos servicios, pero esto dependiendo de los parámetros configurados ya que la consulta a inspecktor se puede apagar y también la validación del correo electrónico.

En caso de que el paramento EncendidoInspektor tenga valor “True” entonces se hace el consumo del api de Inspecktor:
-De la tabla de parámetros se recuperan los siguientes datos que son necesarios para realizar el consumo del api de Inspecktor:

nombre	grupo	key_	valor	estado
Encendido Inspektor	Inspektor	EncendidoInspektor	true	1
URL para autenticarse a API de inspektor	Inspektor	InspektorLogin	http://192.168.0.53:44345/api/Account	1
URL de consumo para API Inspektor	Inspektor	InspektorUrl	http://192.168.0.53:44345/	1
Operacion Inspektor	Inspektor	InspektorOperation	consultainspektor	1
Nombre de usuario para las APIs	APIs	NameUserAPI	credi10web	1
Contraseña del usuario para las APIs	APIs	PasswordAPI	506c5833-f20c-4f22-8466-2f8628257ea1	1

El controlador *InspektorServices.cs* aloja el método del consumo del api que se llama *ConsultaListasRestrictivas()*, este método recibe un *InspektorRequest* que se crea con los datos del prospecto; aquí se utiliza RestClient para realizar las solicitudes HTTP.

```
2 referencias
public class InspektorRequest
{
    [Required]
    1 referencia
    public string NumeroIdentificacion { get; set; }
    [Required]
    1 referencia
    public string Nombre { get; set; }
}
```

Dentro de *ConsultaListasRestrictivas()* se realiza el login a él api teniendo en cuenta el objeto InspektorLoginRequest, este se crea con los parámetros NameUserAPI y PasswordAPI de [*tbl_parametros]* y este objeto es enviado al método *LoginAPIInspektor()* que se encarga de generar el JSON Web Token con el que se podrá realizar la operación “consultainspecktor”.

```
public InspektorResponse ConsultaListasRestrictivas(InspektorRequest request)
{
    var result = new InspektorResponse();
    try
    {
        var responseAccount = LoginAPIInspektor(new InspektorLoginRequest
        {
            Username = ParameterManager.GetParam("NameUserAPI").Value,
            Password = ParameterManager.GetParam("PasswordAPI").Value
        });
        user = "credi10web";
        var client = RestUtil.CreatePostRequest(ParameterManager.GetParam("InspektorUrl").Value, ParameterManager.GetParam("InspektorOperation").Value);
        client.AddHeader("app-user", user);
        client.Authenticate(responseAccount.Data);
        var response = client.Execute(JsonConvert.SerializeObject(request));
        if (response.Completed)
        {
            if (!string.IsNullOrEmpty(response.Data))
            {
                result = JsonConvert.DeserializeObject<InspektorResponse>(response.Data);
            }
            else
            {
                result.Success = false;
                result.Message = "Consulta realizada, respuesta confusa.";
            }
        }
        else
        {
            result.Success = false;
            result.Message = "Tiempo de espera agotado";
            throw new CustomException(result.Message);
        }
```

La respuesta InspektorResponse tiene la siguiente estructura



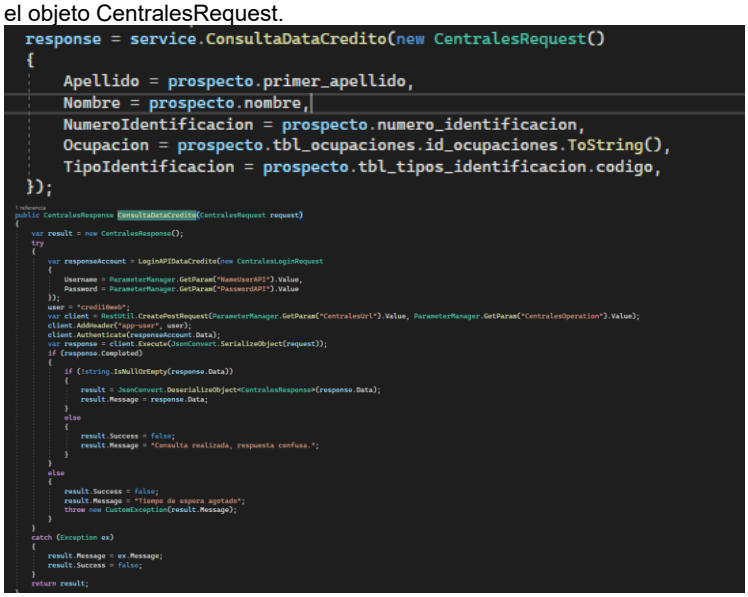
```
public class InspektorResponse
{
    -- referencias
    public InspektorResponse()
    {
        ResponseInspektor = new InspektorResult();
    }
    -- referencias
    public bool Success { get; set; }
    6 referencias
    public string Message { get; set; }
    1 referencia
    public InspektorResult ResponseInspektor { get; set; }
    2 referencias
    public bool IsRestrictive { get; set; }
}
```

Una vez realizada la consulta a inspecktor en la respuesta se valida la bandera IsRestrictive, en caso de que sea true se cambia el estado del prospecto a negado, motivo Listas Restrictivas, en caso de ser false se actualiza el prospecto con el estado estudio, motivo DataCredito, y se continua con la consulta del api de datacredito.

Para el consumo que se realiza de datacredito también se tiene los datos necesarios en *[tbl_parametros]*

nombre	grupo	key_	valor	estado
API Centrales Riesgo	DataCredito	CentralesUrl	http://192.168.0.53:44346/	1
Operacion Centrales	DataCredito	CentralesOperation	centrales	1
URL para autenticarse a API de centrales de riesgo	DataCredito	CentralesLogin	http://192.168.0.53:44346/api/Account	1
Nombre de usuario para las APIs	APIs	NameUserAPI	credi10web	1
Contraseña del usuario para las APIs	APIs	PasswordAPI	506c5833-f20c-4f22-8466-2f8628257ea1	1
URL Centrales de Riesgo	DataCredito	UrlApiCentrales	http://192.168.0.53:44346/centrales/	1

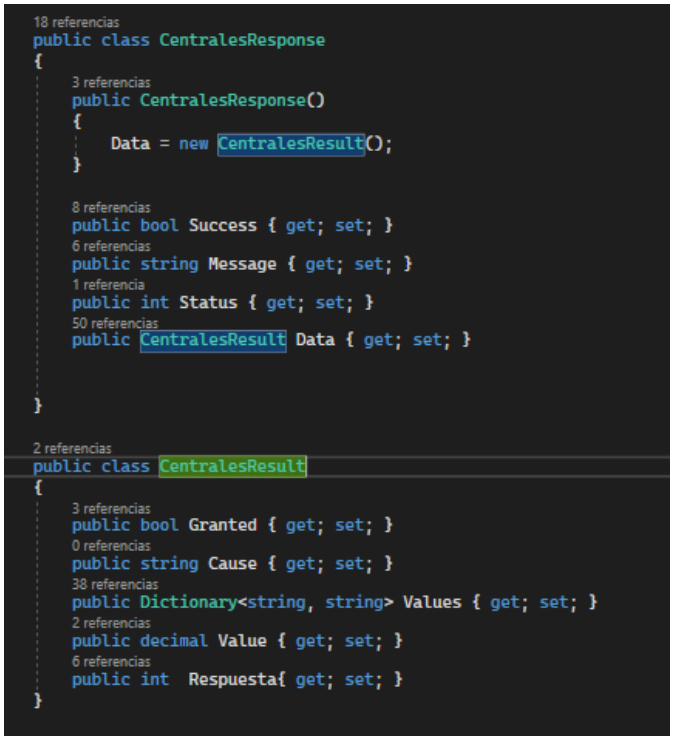
Usando RestClient se consume el login al api de datacredito y una vez logueados con el JSON Web Token se procede a realizar la consulta con los datos del cliente, que están en el objeto CentralesRequest.



```
response = service.ConsultaDataCredito(new CentralesRequest()
{
    Apellido = prospecto.primer_apellido,
    Nombre = prospecto.nombre,
    NumeroIdentificacion = prospecto.numero_identificacion,
    Ocupacion = prospecto.tbl_ocupaciones.id_ocupaciones.ToString(),
    TipoIdentificacion = prospecto.tbl_tipos_identificacion.codigo,
});

1 referencia
public CentralesResponse ConsultaDataCredito(CentralesRequest request)
{
    var result = new CentralesResponse();
    try
    {
        var responseAccount = LoginAPIDataCredito(new CentralesLoginRequest
        {
            Username = ParameterManager.GetParam("NameUserAPI").Value,
            Password = ParameterManager.GetParam("PasswordAPI").Value
        });
        user = "credi10web";
        var client = RestUtil.CreatePostRequest(ParameterManager.GetParam("CentralesUrl").Value, ParameterManager.GetParam("CentralesOperation").Value);
        client.AddHeader("app-user", user);
        client.Authenticate(responseAccount.Data);
        var response = client.Execute(JsonConvert.SerializeObject(request));
        if (response.Completed)
        {
            if (!string.IsNullOrEmpty(response.Data))
            {
                result = JsonConvert.DeserializeObject<CentralesResponse>(response.Data);
                result.Message = response.Data;
            }
            else
            {
                result.Success = false;
                result.Message = "Consulta realizada, respuesta confusa.";
            }
        }
        else
        {
            result.Success = false;
            result.Message = "Tiempo de espera agotado";
            throw new CustomException(result.Message);
        }
    }
    catch (Exception ex)
    {
        result.Message = ex.Message;
        result.Success = false;
    }
    return result;
}
```

El objeto CentralesReponse contiene la respuesta de la consulta a DataCredito en el parámetro data que es un objeto CentralesResult



```
18 referencias
public class CentralesResponse
{
    3 referencias
    public CentralesResponse()
    {
        Data = new CentralesResult();
    }

    8 referencias
    public bool Success { get; set; }
    6 referencias
    public string Message { get; set; }
    1 referencia
    public int Status { get; set; }
    50 referencias
    public CentralesResult Data { get; set; }


}

2 referencias
public class CentralesResult
{
    3 referencias
    public bool Granted { get; set; }
    0 referencias
    public string Cause { get; set; }
    38 referencias
    public Dictionary<string, string> Values { get; set; }
    2 referencias
    public decimal Value { get; set; }
    6 referencias
    public int  Respuesta{ get; set; }
}
```

Para evaluar si la solicitud fue aprobada se validan dos cosas: que la propiedad de tipo entero Respuesta de CentralesResult sea 13, y que la propiedad Granted sea true, vea las posibles respuestas en el manual de datacredito en el siguiente link Manual de Implementación Web Service HC+PN Versión 1 5 6 WSS.pdf

si la Respuesta es diferente de 13 y también es diferente de 0 entonces se actualiza el estado del prospecto a Negado, motivo DataCredito, si la respuesta es 0 entonces no se actualiza el prospecto, se envía el correo de rechazado alojado en *[tbl_configuracion_mensajeria]* con el key_ *'EmailMessageDeclined'*.

Si la respuesta de CentralesResult es 13, y la propiedad Granted es true, entonces se procede a extraer la información retornada por datacredito y se guarda la información del cliente en la tabla *[tbl_calificacion]*. De CentralesReponse se extrae la información de la persona consultada usando el método extraer información, esto con el fin realizar tres validaciones cruciales para aprobar el cupo, las validaciones son las siguientes: que el celular de registro coincida con uno de los números de celular que retorna Datacredito, que la fecha de expedición coincida con la retornada por Datacredito y que el correo coincida con alguno de los retornados por Datacredito, cabe resaltar que la validación del correo electrónico se puede encender y apagar a partir del parámetro *'EncendidoValidacionCorreo'*, si el valor es true se realiza la validación del correo si es false se omite esta validación.

2	91	Encendido validacion correo electronico originacion	Originacion	EncendidoValidacionCorreo	True	1	Dev

En caso de que alguna de las tres validaciones de datos no se cumpla entonces se asigna el estado del prospecto según la validación que no se haya cumplido



```
                else
                {
                    ActualizarEstadoProspecto(prospecto, Constant.MotivosProspecto.NoCoincideCorreo, (int)EstadosProspecto.Negado);
                    responseCentralesInspektor.Aprobado = false;
                    responseCentralesInspektor.Completed = true;
                    var mensaje = ParameterManager.GetParamsEs(Constant.MensajesKey.InfomaSolicitudRechazada);
                    responseCentralesInspektor.Mensaje = mensaje.Value;
                    responseCentralesInspektor.MensajeCode = mensaje.Codigo;
                }
            }
            else
            {
                ActualizarEstadoProspecto(prospecto, Constant.MotivosProspecto.NoCoincideFechaExpedicion, (int)EstadosProspecto.Negado);
                responseCentralesInspektor.Aprobado = false;
                responseCentralesInspektor.Completed = true;
                var mensaje = ParameterManager.GetParamsEs(Constant.MensajesKey.InfomaSolicitudRechazada);
                responseCentralesInspektor.Mensaje = mensaje.Value;
                responseCentralesInspektor.MensajeCode = mensaje.Codigo;
            }
        }
        else
        {
            ActualizarEstadoProspecto(prospecto, Constant.MotivosProspecto.NoCoincideCelular, (int)EstadosProspecto.Negado);
            responseCentralesInspektor.Aprobado = false;
            responseCentralesInspektor.Completed = true;
            var mensaje = ParameterManager.GetParamsEs(Constant.MensajesKey.InfomaSolicitudRechazada);
            responseCentralesInspektor.Mensaje = mensaje.Value;
            responseCentralesInspektor.MensajeCode = mensaje.Codigo;
        }
```

Si las tres validaciones se cumplen entonces se procede con la creación del cliente y el cupo según la calificación y el monto aprobado por datacredito; para realizar esto último se hace uso del método CrearCliente de OriginacionController que se va a encargar de crear las direcciones, celulares, correos electrónicos que retorno datacredito en las tablas: *[tbl_cliente_direcciones], [tbl_cliente_telefonos], [tbl_cliente_correos]* respectivamente, luego de crear estos datos se realiza el registro en la tabla *[tbl_clientes]* y por último se usa el método *CrearCupo* de *CuposController* para crear el cupo asociado al cliente, el histórico de estados del cupo, y el histórico de montos del cupo.

Hasta este punto el cupo ha sido generado y se ve de la siguiente forma, donde cada registro tiene un token único en el campo token_aceptacion.



id_cupo	fecha_creacion	fk_id_cliente	monto_aprobado	dia_corte	numero_refinanciaciones	fk_id_estado	aceptado	token_aceptacion	otp_aceptacion	fecha_aceptacion	ip_aceptacion	usuario_modificacion	fecha_modificacion
134	2019-06-12 00:00:00.000	134	360000.00	NULL	0	10	0	eWr0x.YHGSajCfk1hKar196u/GEi9HMU	NULL	NULL	NULL	NULL	NULL
150	2019-06-15 00:00:00.000	150	1200000.00	NULL	0	10	0	aGLaDNI79PLeXepJKBqhGWuWa7	NULL	NULL	NULL	NULL	NULL
280	2019-06-29 00:00:00.000	281	300000.00	NULL	0	10	0	XRC2SSQ2n.VKBkUfy27aNETpCCJR.P	NULL	NULL	NULL	NULL	NULL
296	2019-07-01 00:00:00.000	300	4000000.00	NULL	0	10	0	PF^Itqp._BDCMMkaw@aF.ZSzILFN	NULL	NULL	NULL	NULL	NULL
446	2019-07-06 00:00:00.000	455	1200000.00	NULL	0	10	0	HeJfeiTMy..WDC1k3vdvLB5uT1G1Oy.	NULL	NULL	NULL	NULL	NULL
467	2019-07-07 00:00:00.000	476	360000.00	NULL	0	10	0	em'HUDJ5Oswqfv/z3.nHB6KFLIB	NULL	NULL	NULL	NULL	NULL
521	2019-07-09 00:00:00.000	533	480000.00	NULL	0	10	0	fIvaOG2Y8dPpdUnazLUOxH5zBDkZH	NULL	NULL	NULL	NULL	NULL

Dentro de *CrearCupo* después de crear el cupo, se llama el método *GenerarOtpAceptacionCupo* que se encarga de generar un OTP registrarlo en la tabla *[tbl_otp_aceptacion]* y también de enviar el correo electrónico con el link único de aceptación de cupo y el OTP que se registró, el link único conserva la siguiente forma https://www.credi10.com.co/originacion/aceptacioncupo/validartokenaceptacion?token=0a40d4e834604c3d99d32b76861a4fac

Las plantillas html de los correos que se envían en el proceso de originacion están almacenadas en la tabla *[tbl_configuracion_mensajeria]*

id_configuracion_mensajeria	descripcion_tipo_envio	grupo_parametros	key_	mensaje
1	Html para envio del token de solicitud de credito	Email	EmailMessageSolicitud	<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Tra...
3	Html para envio de aprobacion del cupo	Email	EmailMessageApproved	<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Tra...
4	Html para envio de cupo rechazado	Email	EmailMessageDeclined	<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Tr...

1.3.5 Log

Durante el proceso de originacion se registra en la tabla *[tbl_logs]* las diferentes acciones que realiza el prospecto para la obtención de su cupo, la columna acción almacena la descripción del punto en el proceso de originacion en el que se encuentra el prospecto y el

detalle almacena la información relacionada al resultado de la acción, las acciones que corresponden a la originacion se muestran a continuación en la siguiente tabla

accion
Crear Prospecto (Admin)
Validacion OTP Prospecto (Admin)
Aceptacion HabeasData Prospecto (Admin)

El seguimiento a las anteriores acciones puede ser individual por cada prospecto mediante las siguientes consultas:

- Crear Prospecto (Admin)
  SELECT * FROM tbl_logs WHERE accion = 'Crear Prospecto (Admin)' and detalle like '%<cedulaProspecto>%';

- Validacion OTP Prospecto (Admin)
  SELECT * FROM tbl_logs WHERE accion = 'Validacion OTP Prospecto (Admin)' and detalle like '%<correo de tbl_prospectos>%';

- Aceptacion HabeasData Prospecto (Admin)
  SELECT * FROM tbl_logs WHERE accion = 'Aceptacion HabeasData Prospecto (Admin)' and detalle like '%<token de tbl_prospectos>%';

1.3.5 Lista de Scripts y la descripción de lo que contiene cada uno

El script de creación de la base de datos: Script Creacion de la Base de Datos